

Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 24**

① Kruskal Algorithm :-

MST - Kruskal (G, w)

1. $A \leftarrow \phi$
2. for each vertex $v \in V[G]$
3. do **MakeSet**(v)
4. Sort the edges of E into non-decreasing order by weight w
5. for each edge $(u, v) \in E$, in order by non-decreasing weight
6. do if **Find-Set**(u) $\neq$ **Find-Set**(v) #Avoid cycle #Both have same parent
7. then $A \leftarrow A \cup \{(u, v)\}$
8. **UNION**(u, v)
9. return A

* **Make-set**(x) creates a new set whose only member is x . Since the set are disjoint, we requires that x not already be in some other set.

* **Find-set**(x) returns a pointer to the representative of the unique set containing x .

$(h, g) \rightarrow 1$

$(g, f) \rightarrow 2$

$(a, b) \rightarrow 4$

$(i, g) \rightarrow 6$

$(c, d) \rightarrow 7$

$(h, i) \rightarrow 7$

$(a, h) \rightarrow 8$

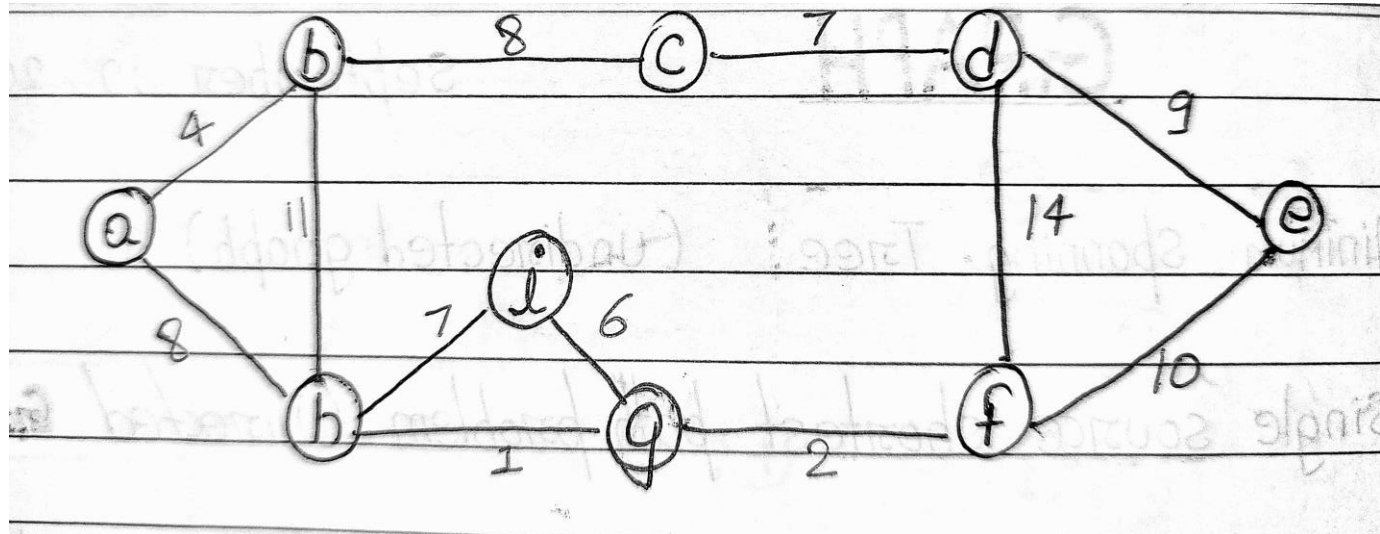
$(b, c) \rightarrow 8$

$(d, e) \rightarrow 9$

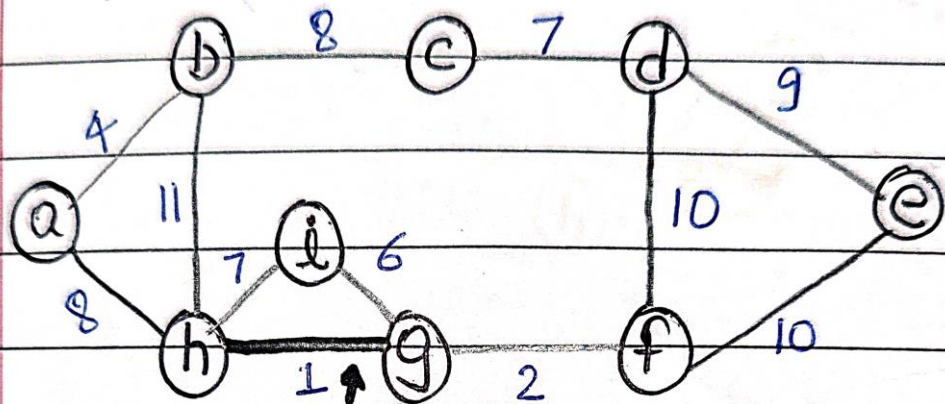
$(e, f) \rightarrow 10$

$(b, h) \rightarrow 11$

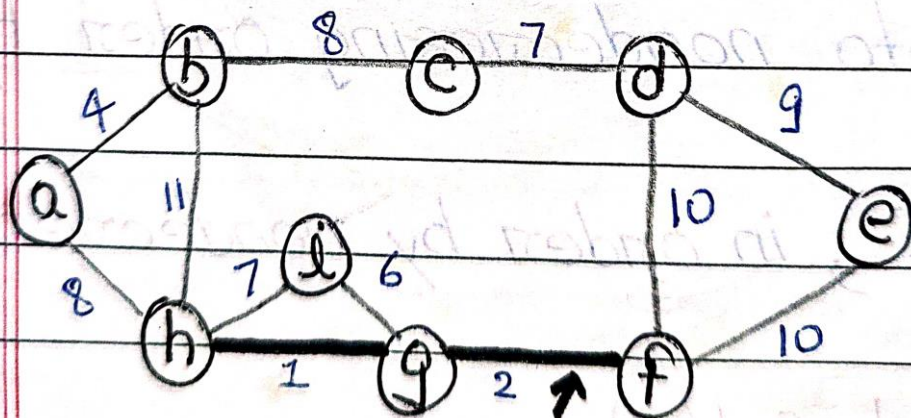
$(d, f) \rightarrow 14$



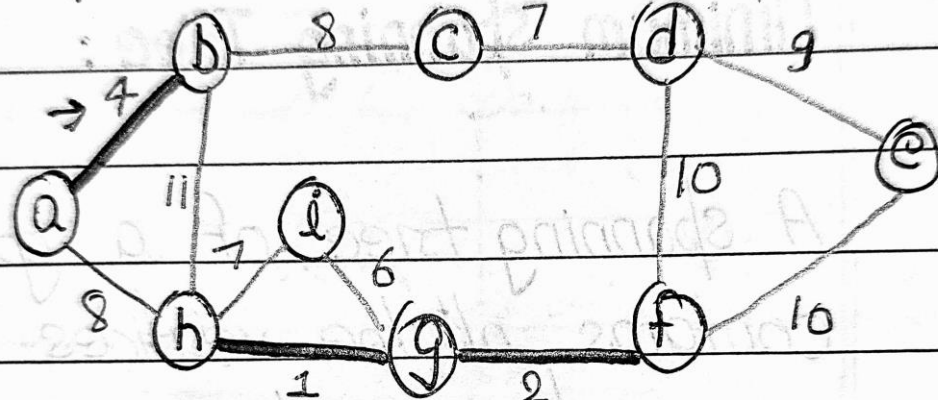
step:1



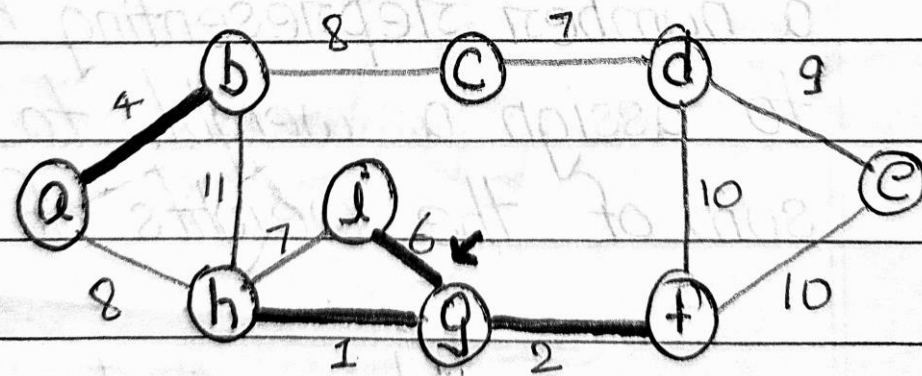
Step:2



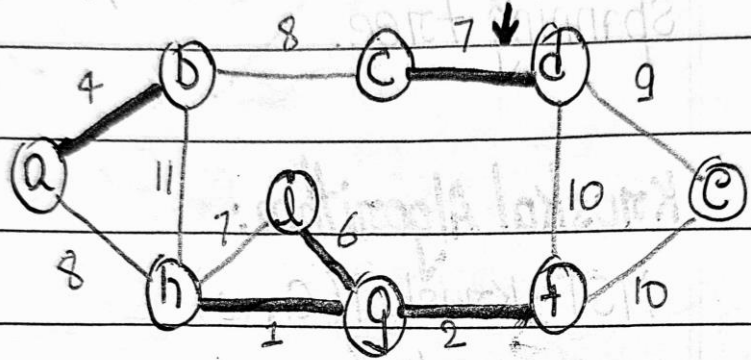
Step:3



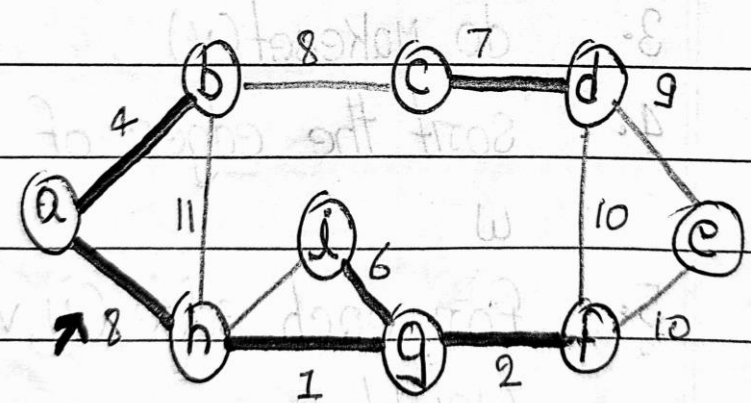
Step:4



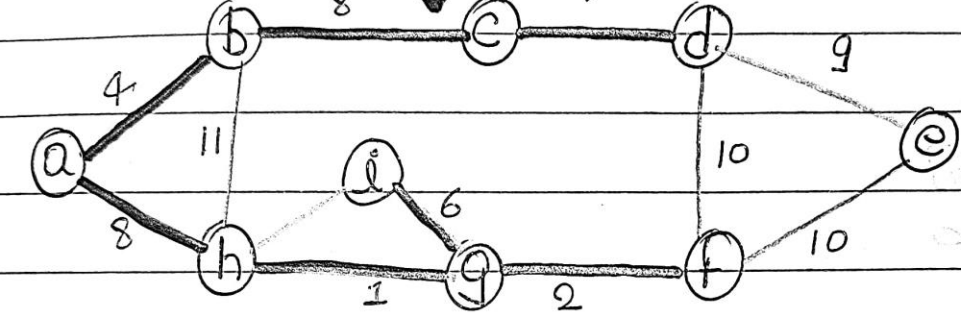
step:5



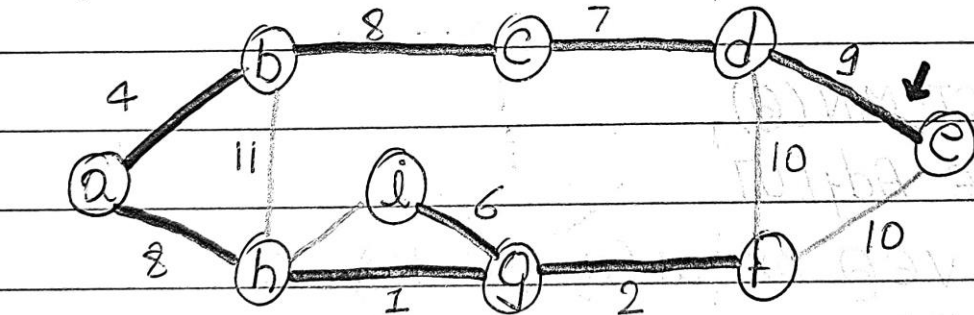
step:6



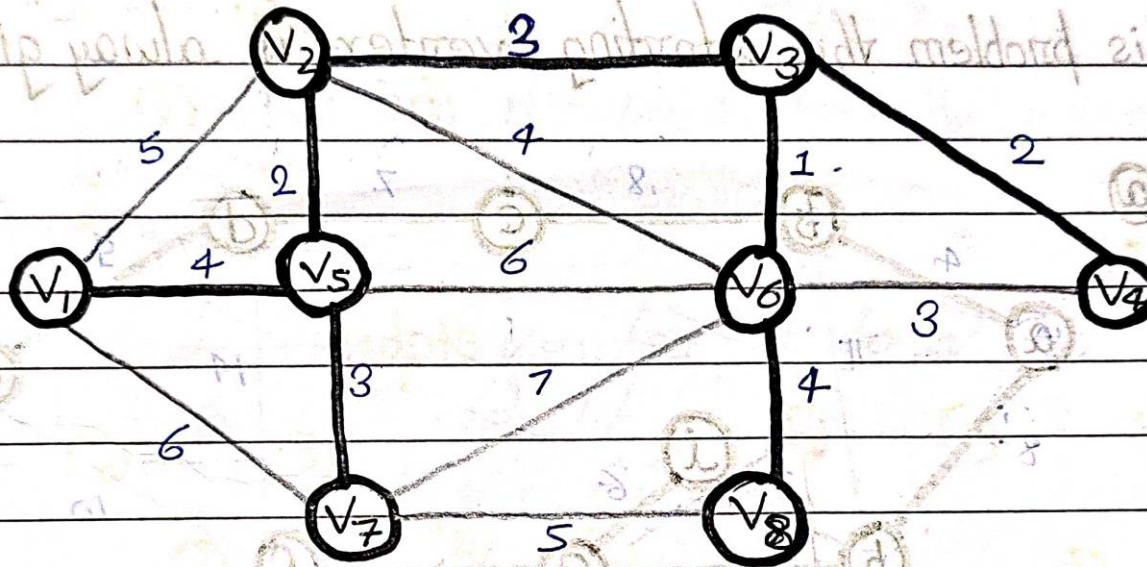
step:7



step:8



Total weight = $1 + 2 + 4 + 6 + 7 + 8 + 8 + 9 = 45$ Ans



$$(V_3, V_6) = 1$$

$$(V_1, V_2) = 5$$

$$(V_2, V_5) = 2$$

$$(V_7, V_8) = 5$$

$$(V_3, V_4) = 2$$

$$(V_5, V_6) = 6$$

$$(V_2, V_3) = 2$$

$$(V_1, V_7) = 6$$

$$(V_5, V_7) = 3$$

$$(V_7, V_6) = 7$$

$$(V_6, V_4) = 3$$

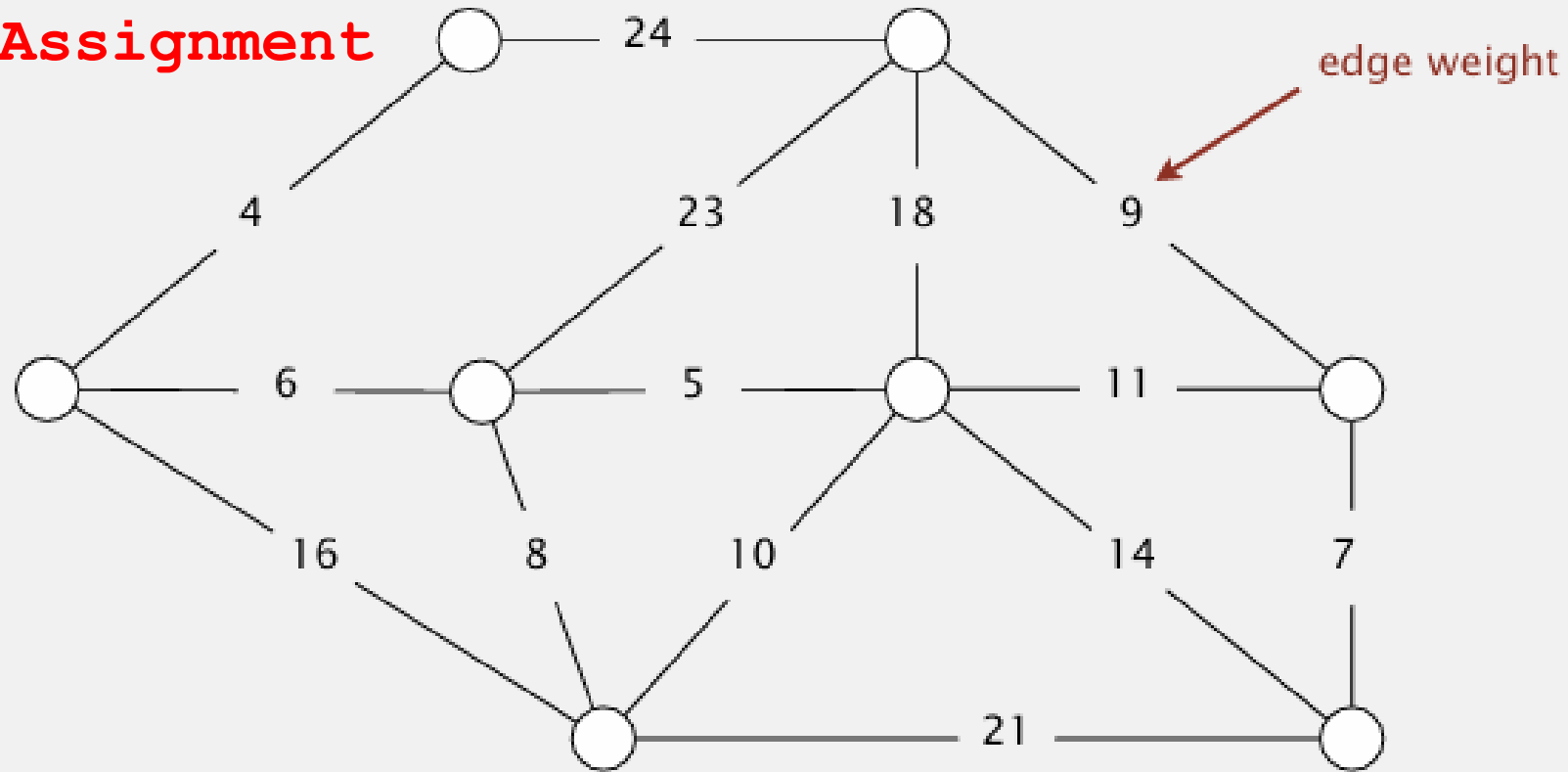
$$(V_1, V_5) = 4$$

$$(V_2, V_6) = 4$$

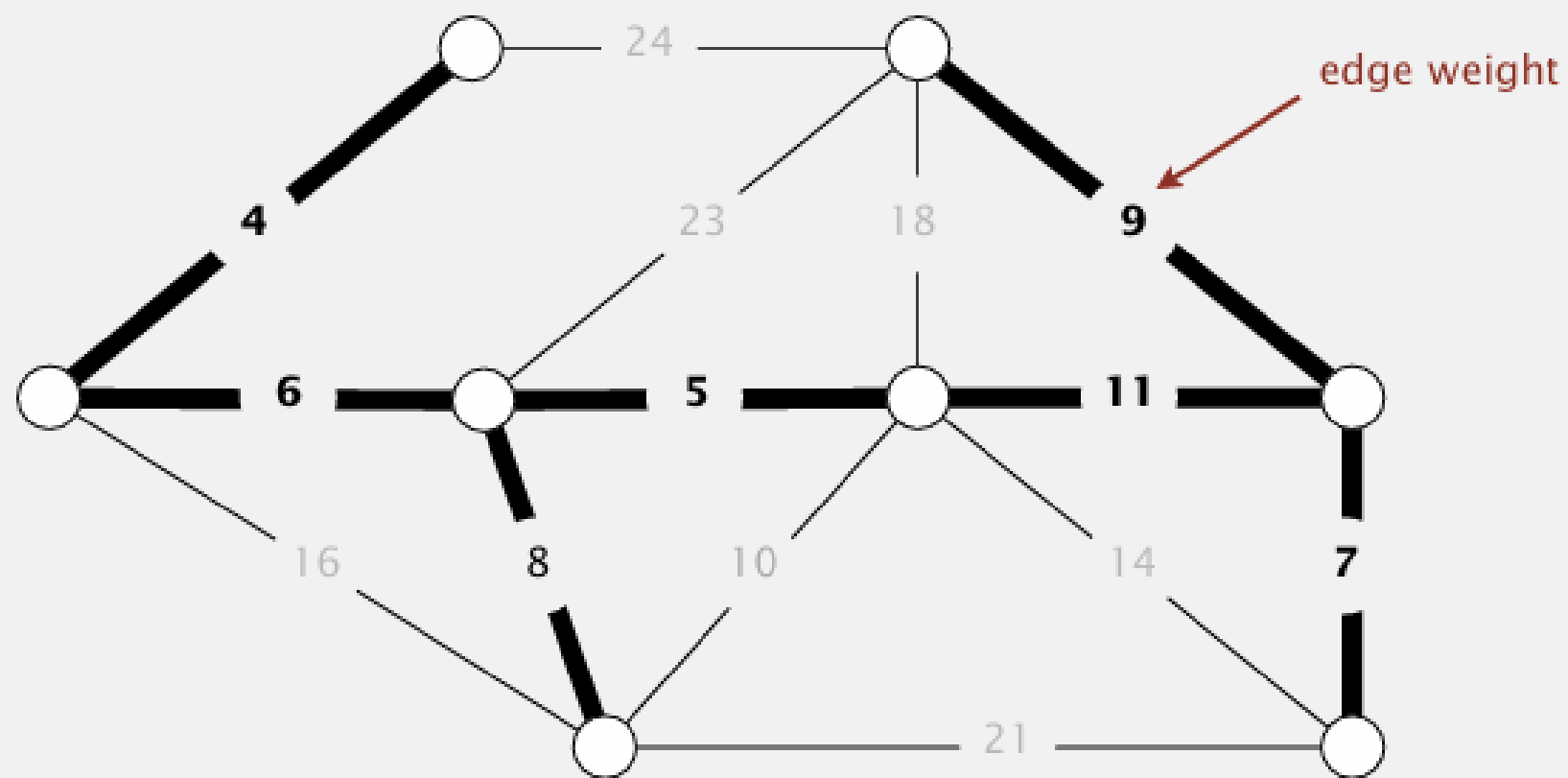
$$(V_6, V_8) = 4$$

weight = 18.

Class Assignment



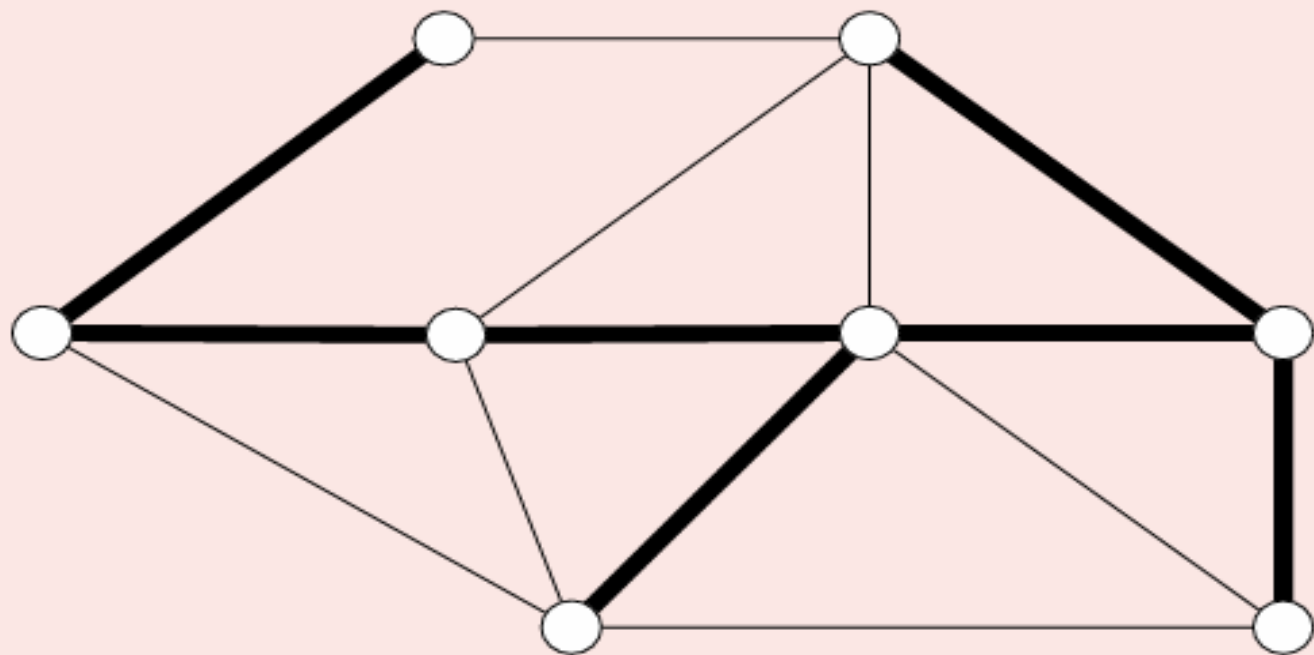
edge-weighted graph G



minimum spanning tree T

Let T be any spanning tree of a connected graph G with V vertices. Which of the following properties must hold?

- A. T contains exactly $V - 1$ edges.
- B. Removing any edge from T disconnects it.
- C. Adding any edge to T creates a cycle.
- D. All of the above.



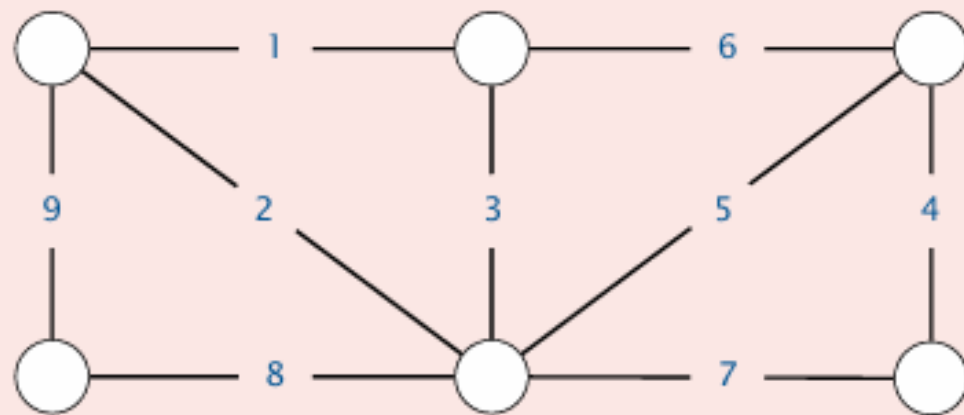
In which order does Kruskal's algorithm select edges in MST?

A. 1, 2, 4, 5, 6

B. 1, 2, 4, 5, 8

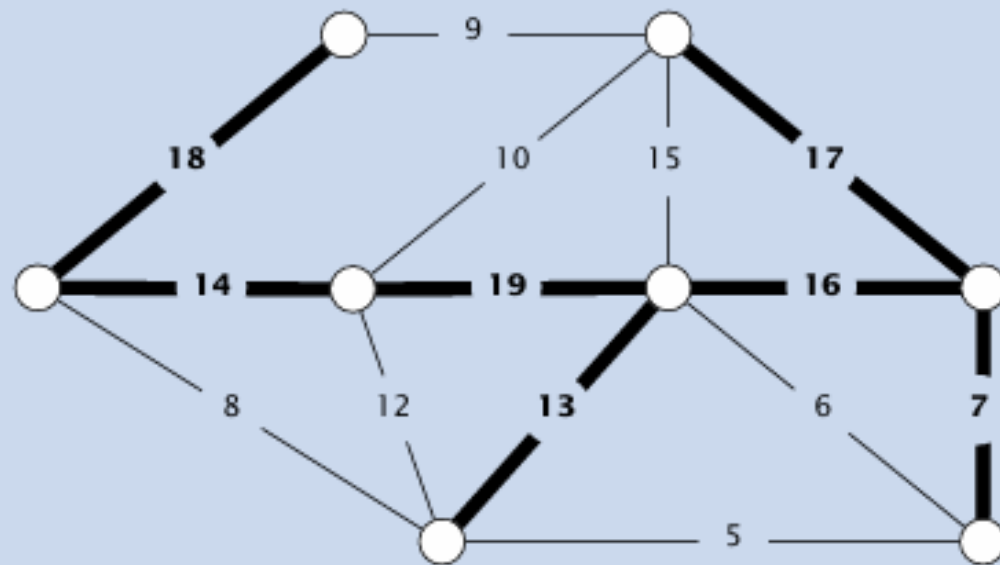
C. 1, 2, 5, 4, 8

D. 8, 2, 1, 5, 4



Problem. Given an undirected graph G with positive edge weights, find a spanning tree that **maximizes the sum** of the edge weights.

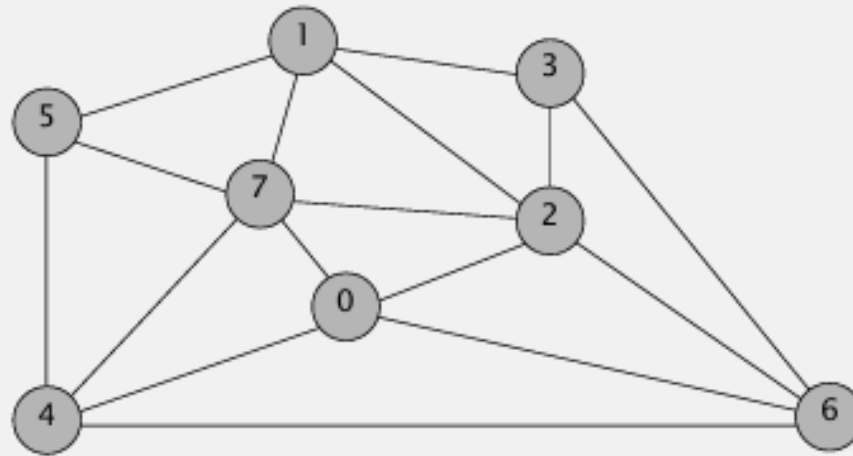
Goal. Design algorithm with $E \log E$ running time.



maximum spanning tree T (weight = 104)

Prim's algorithm

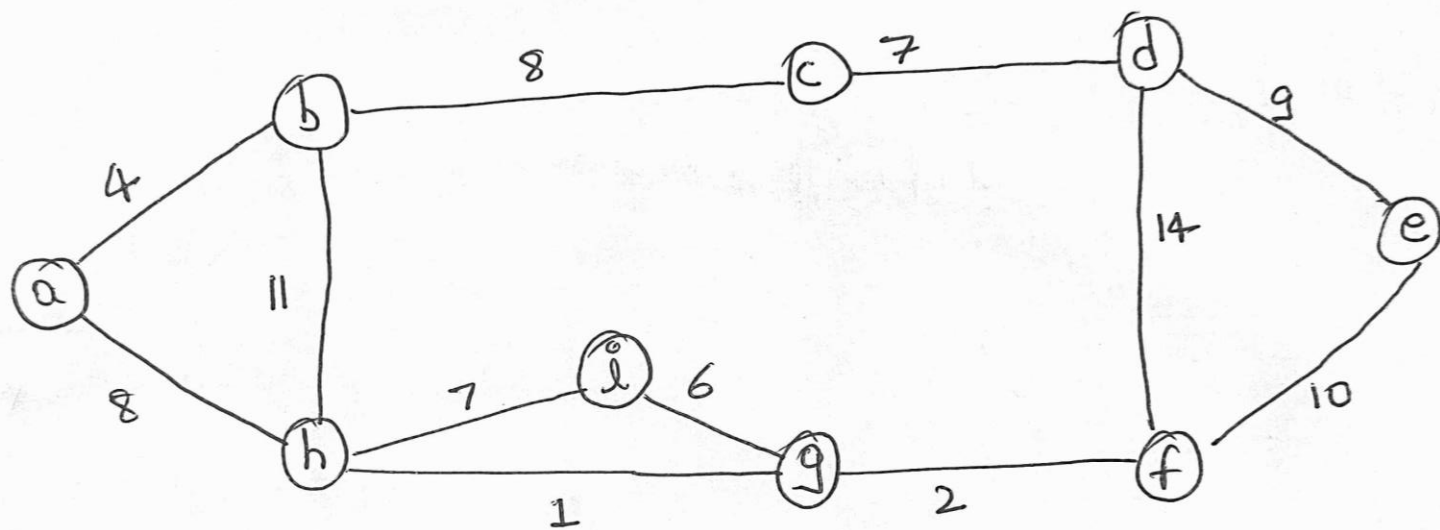
- Start with vertex 0 and grow tree T .
- Repeat until $V - 1$ edges:
 - add to T the min-weight edge with exactly one endpoint in T



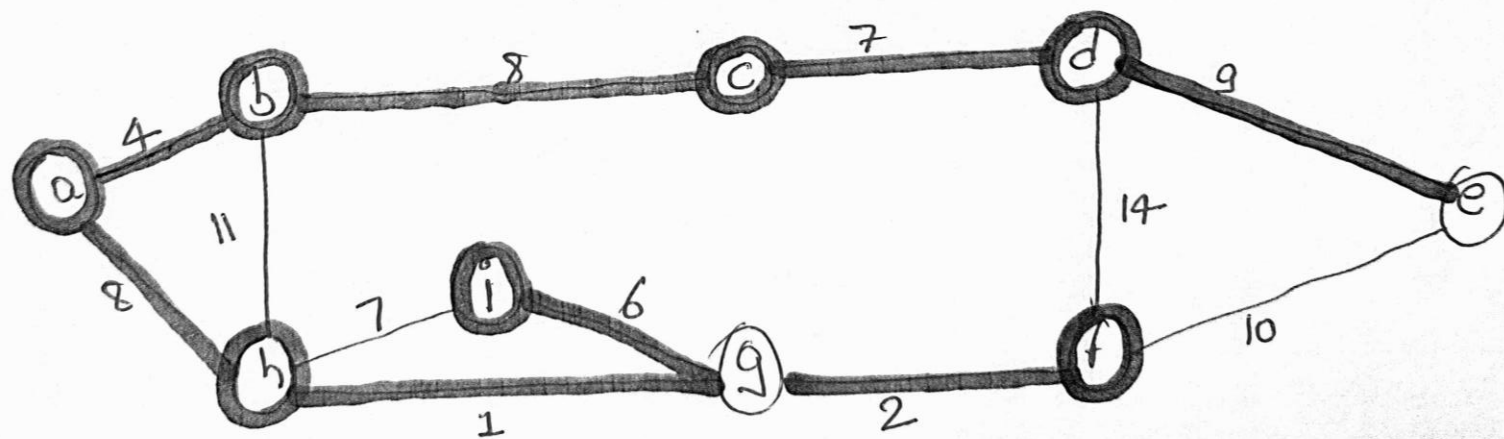
an edge-weighted graph

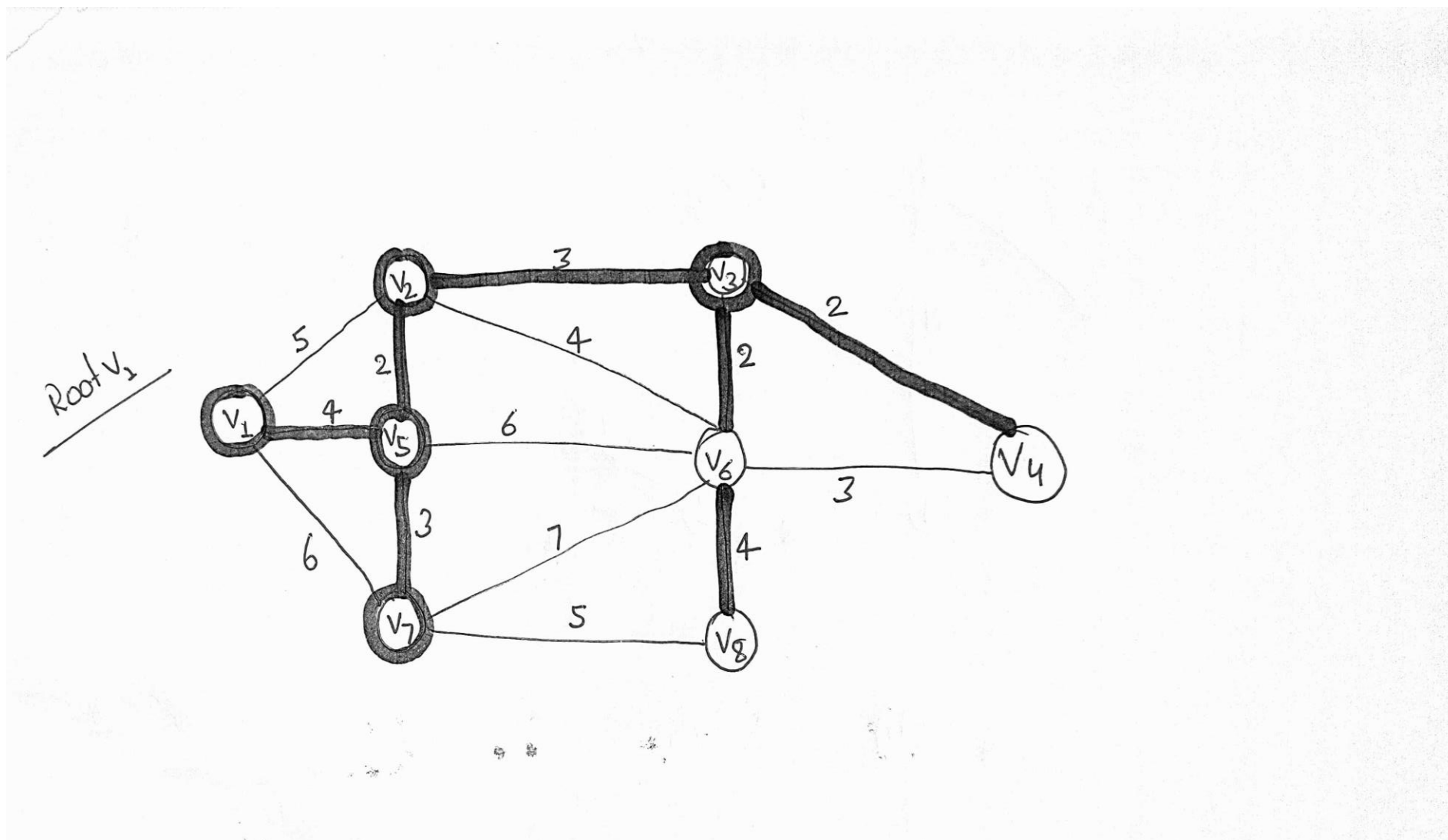
0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93

Root(a)



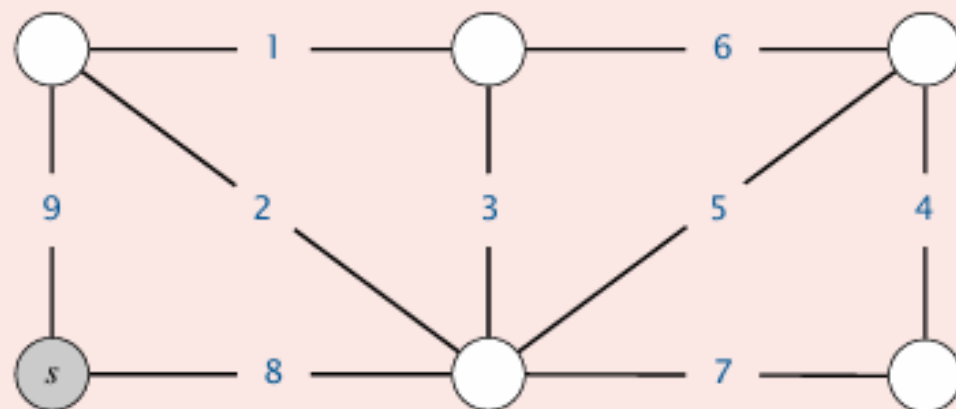
Solution





In which order does Prim's algorithm select edges in the MST?
Assume it starts from vertex s .

- A. 1, 2, 3, 4, 5
- B. 1, 2, 5, 4, 8
- C. 8, 2, 1, 5, 4
- D. 8, 5, 4, 2, 1



■ Single - Source Shortest Paths :-

- > Suppose that you need to drive from Patna, Varanasi, to Delhi, Chandigarh, by the shortest possible route.
 - > Your GPS contains information about the entire road network of the India, including the road distance between each pair of adjacent intersections.
 - > How can your GPS determine this shortest route?
 - > Your GPS would need to examine an enormous number of possibilities, most of which are simply not worth considering.
- * The goal is to find a shortest path from a given intersection in different section.

SINGLE SOURCE Shortest Path Problem:

In single source shortest path problem we are given a weighted directed graph $G=(V,E)$ with weight function $w: E \rightarrow \mathbb{R}$ mapping edges to real valued weights. The weight of the path is the sum of the weights of its constituent edges.

$$w: E \rightarrow \mathbb{R}$$

$$p = (v_0, v_1, \dots, v_k)$$

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

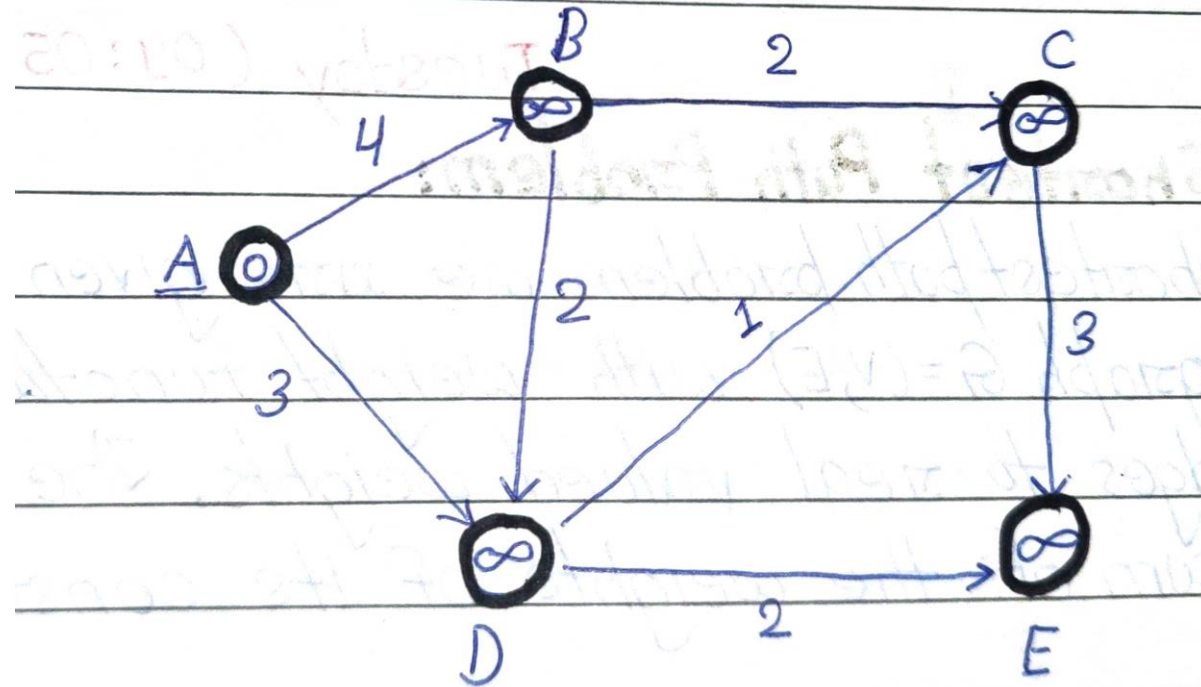
we define shortest path weight from u to v by

$$\delta(u, v) = \begin{cases} \min \{ w(p) : u \xrightarrow{\text{via path}} v \text{ if there is a path from } u \text{ to } v \\ \infty \text{ otherwise} \end{cases}$$

A shortest path from vertex u to v is then defined as any path p with weight $w(p) = \delta(u, v)$

Initialize Single Source (G, s)

1. for each vertex $v \in V[G]$
2. do $d[v] \leftarrow \infty$
3. $\pi[v] \leftarrow \text{NIL}$
4. $d[s] \leftarrow 0$



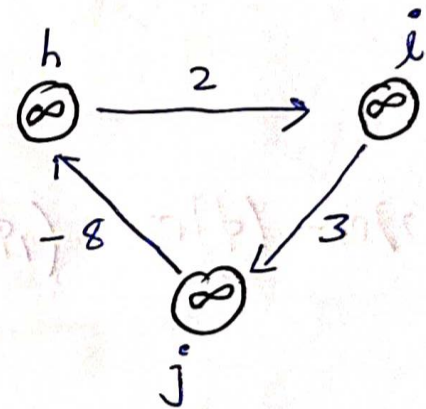
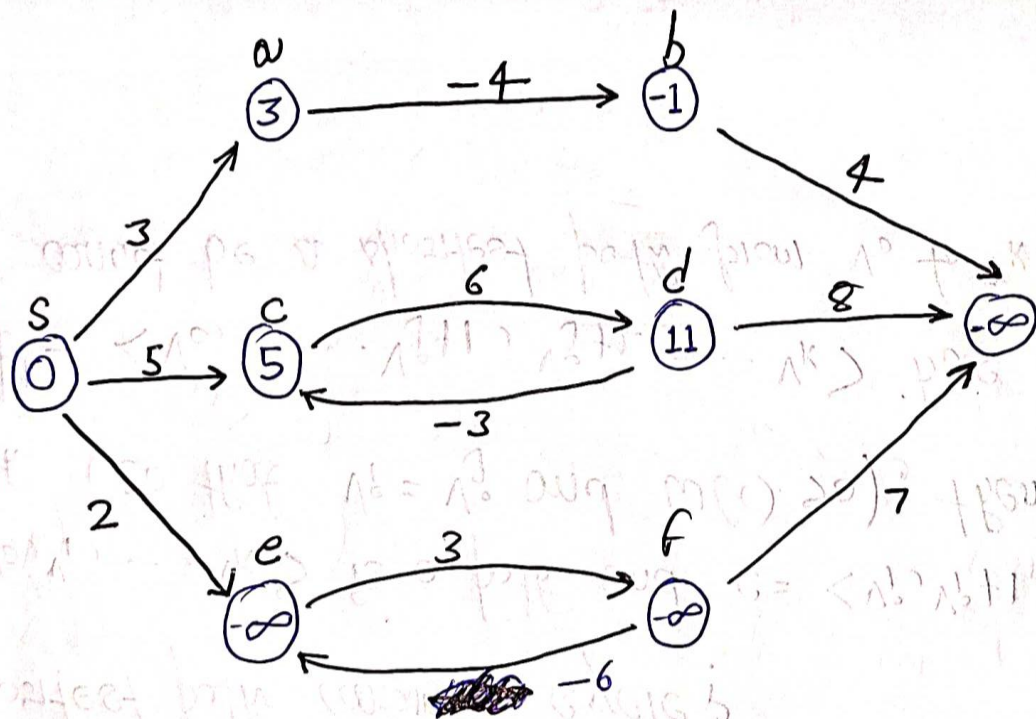
Variants

- 1) Single-destination shortest-paths problem
- 2) Single-pair shortest-path problem : Bellman Ford & Dijkstra
- 3) All-pairs shortest-path problem : Floyd Warshall

Lemma: (Subpaths of shortest paths are shortest paths)
Given a weighted, directed graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be the shortest path from vertex v_0 to vertex v_k and, for any i and j such that $0 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to vertex v_j . Then p_{ij} is a shortest path from v_i to v_j .

* Negative-weight edges :-

- * Some instances of the single-source shortest-paths problem may include edges whose weights are negative.
- * If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s , then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.
- * If the graph contains a negative-weight cycle reachable from s , however, shortest path weights are not well defined.
- If there is a negative-weight cycle on some path from s to v , we define $\delta(s, v) = -\infty$.



* There is only one path from s to $a \rightarrow \text{path } \langle s, a \rangle$, we have $\delta(s, a) = w(s, a) = 3$

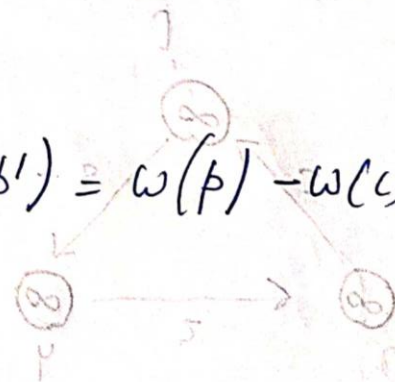
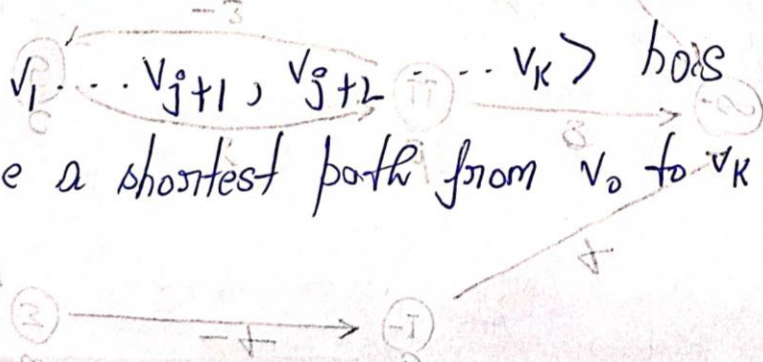
* There is only one path from s to $b \rightarrow \delta(s, b) = 3 + (-4) = -1$

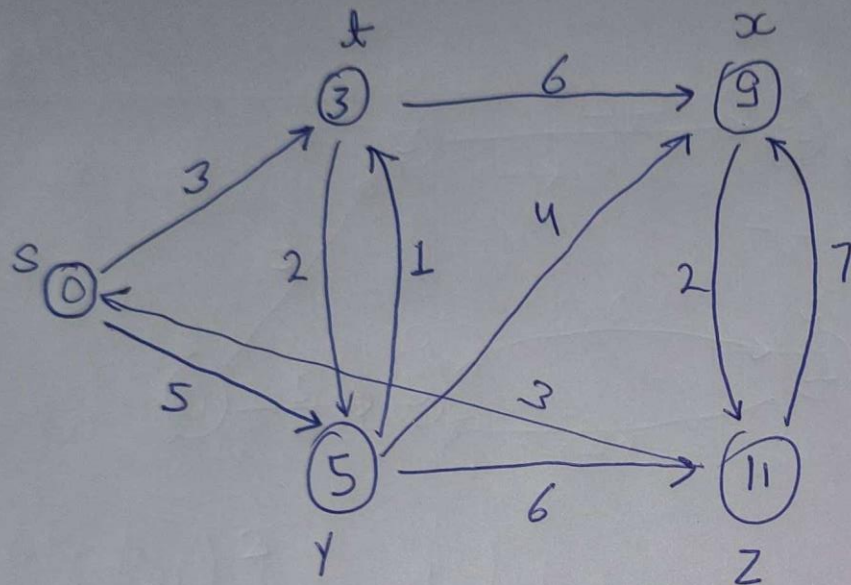
* Infinitely many paths from s to c : $\langle s, c \rangle, \langle s, c, d, c \rangle, \langle s, c, d, c, d, c \rangle, \dots$

- * Some shortest-path algorithms, such as Dijkstra's algorithm, assume that all edge weights in the input graph are nonnegative, as in a road network.
- * Bellman-Ford algorithm, allows negative-weight edges in the input graph and produce a correct answer as long as no negative-weight cycles are reachable from the source.

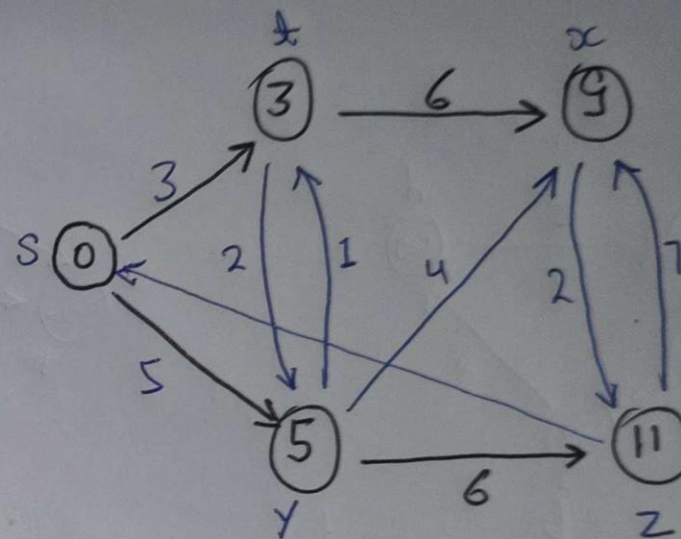
⇒ Cycles

- can a shortest path contain cycle?
- if $p = \langle v_0, v_1, \dots, v_k \rangle$ is a path and $c = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a positive-weight cycle on this path (so that $v_i = v_j$ and $w(c) > 0$), then
- the path $p' = \langle v_0, v_1, \dots, v_{j+1}, v_{j+2}, \dots, v_k \rangle$ has weight $w(p') = w(p) - w(c) < w(p)$ and so p cannot be a shortest path from v_0 to v_k .



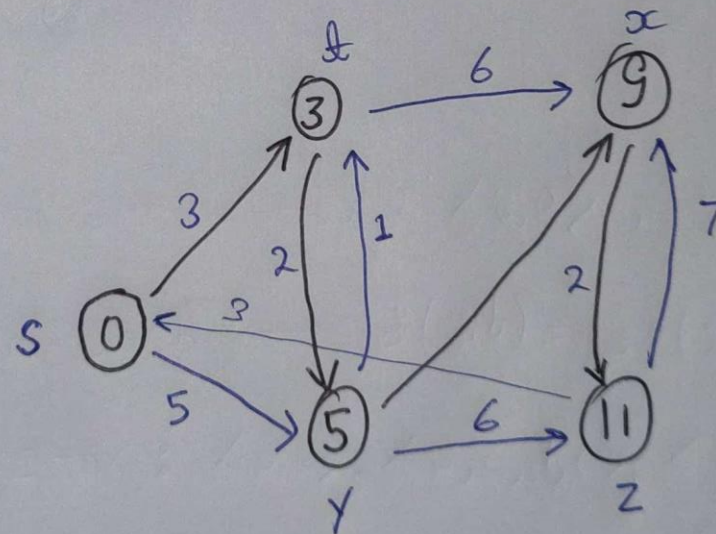


(a)



(b)

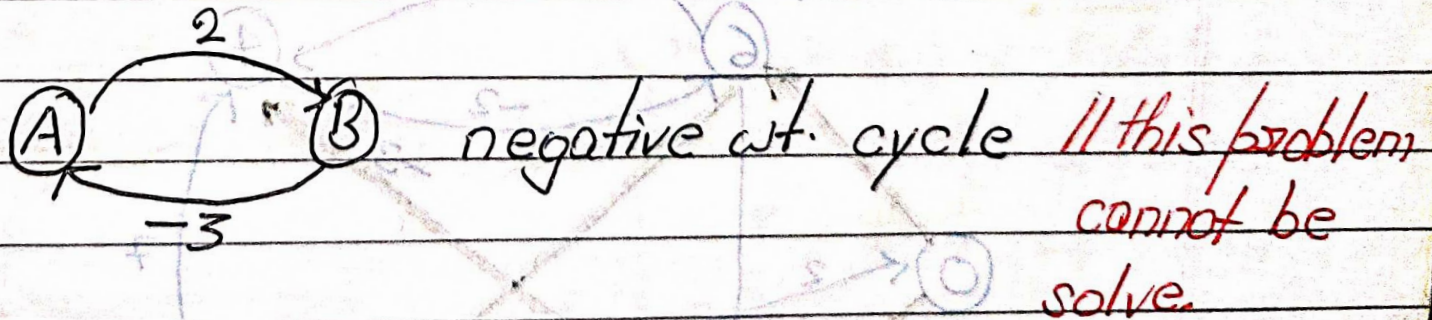
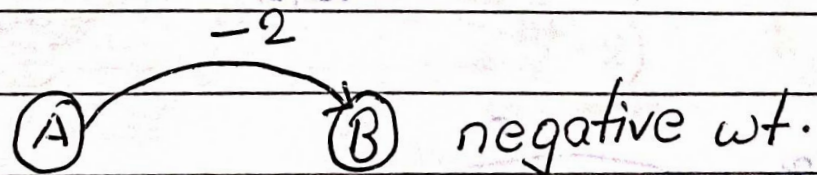
①



②

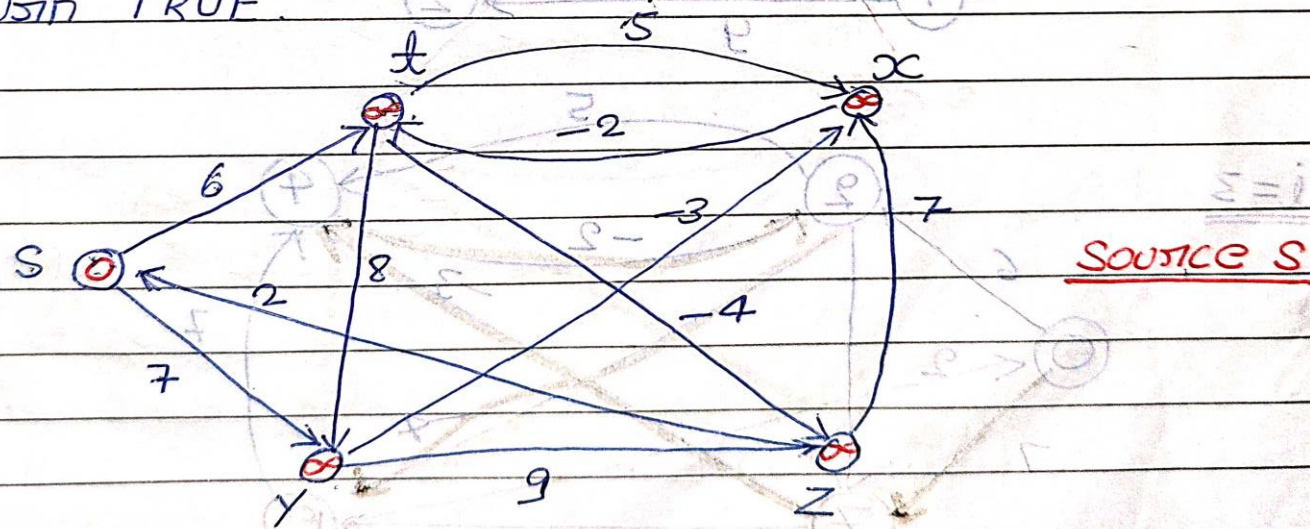
Bellman-Ford Algorithm:

The Bellman Ford Algorithm solve the single source shortest path problem in general case in which edge weight may negative. It return boolean value which is either true / false. It Return true in solvable problem i.e; there is no negative wt cycle otherwise false.



Bellman-Ford (G, w, s)

1. Initialize-Single-Source (G, s)
2. for $i \leftarrow 1$ to $|V[G]| - 1$
3. do for each edge $(u, v) \in E[G]$
4. do Relax (u, v)
5. for each edge $(u, v) \in E[G]$
6. do if $d[v] > d[u] + w(u, v)$
7. then return FALSE
8. return TRUE.

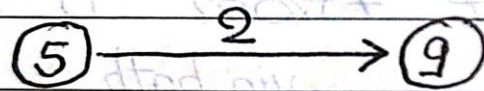


• **Relax** (u, v, w)

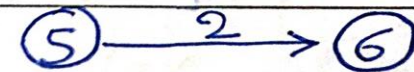
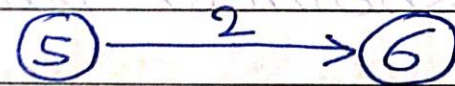
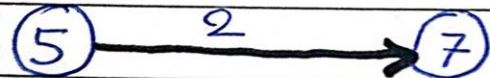
1. if $d[v] > d[u] + w(u, v)$

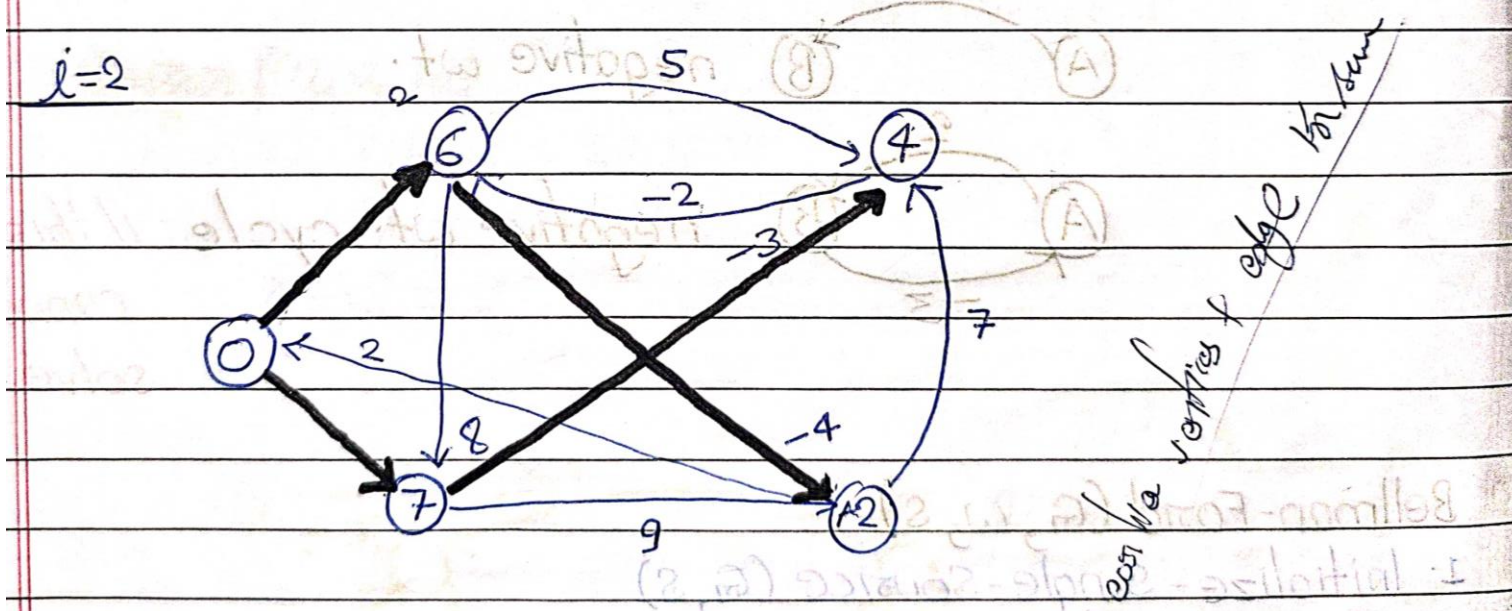
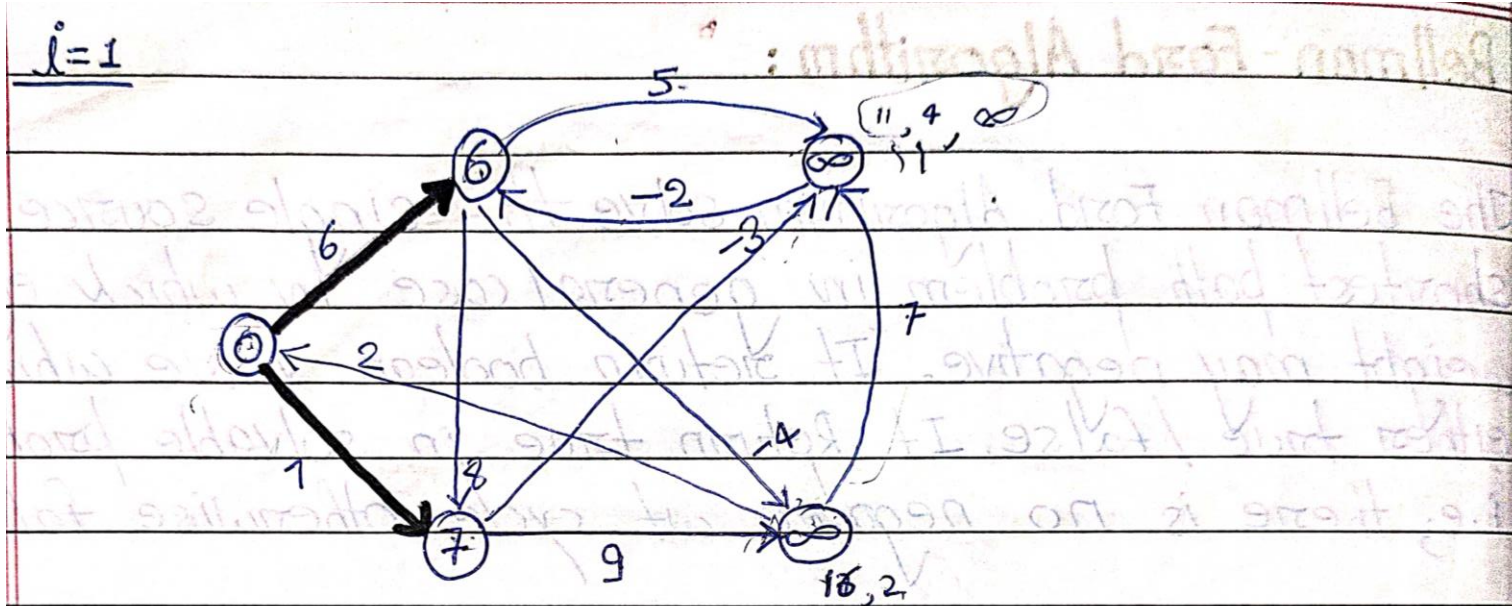
then $d[v] \leftarrow d[u] + w(u, v)$

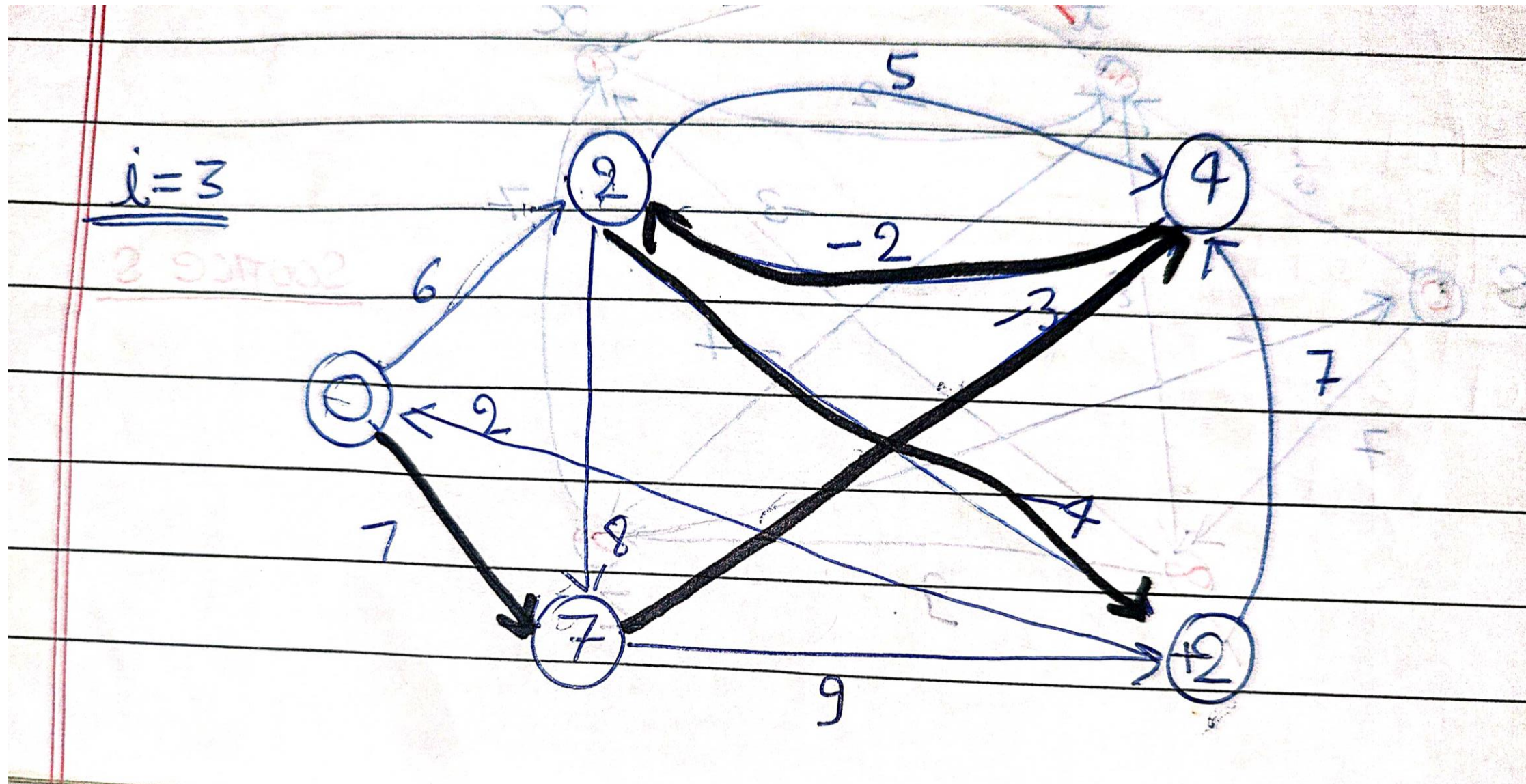
$\pi[v] \leftarrow u$



Relax

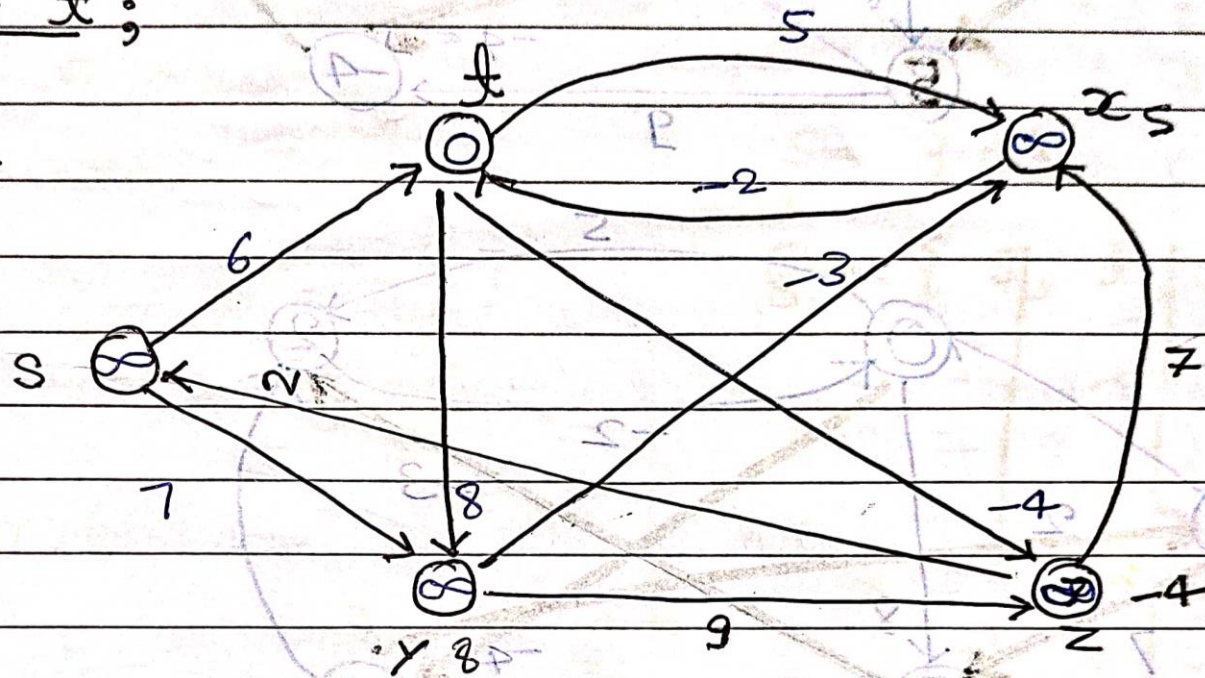




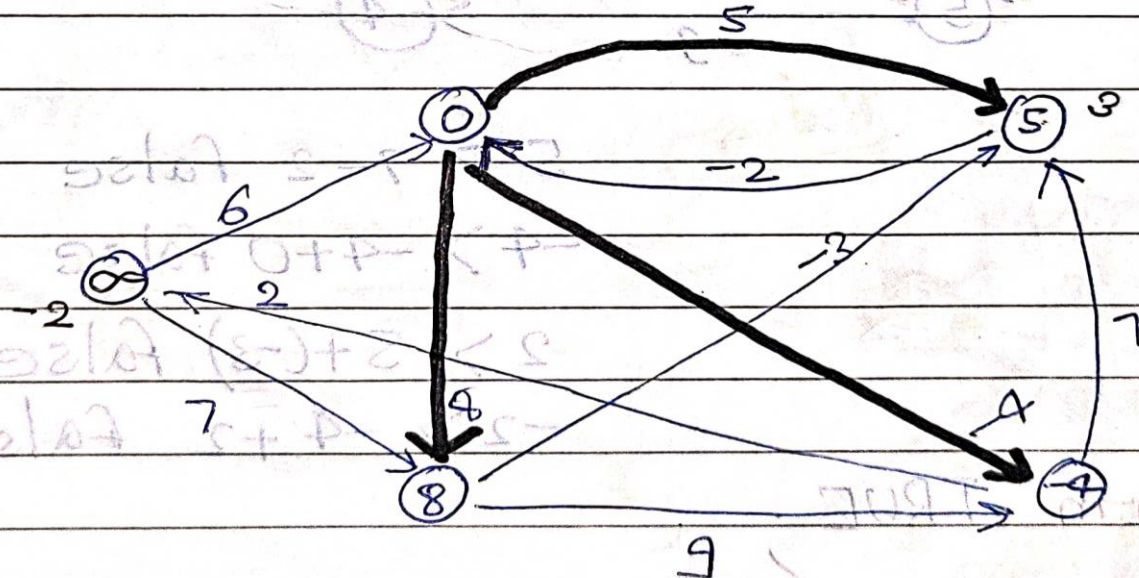


Source t;

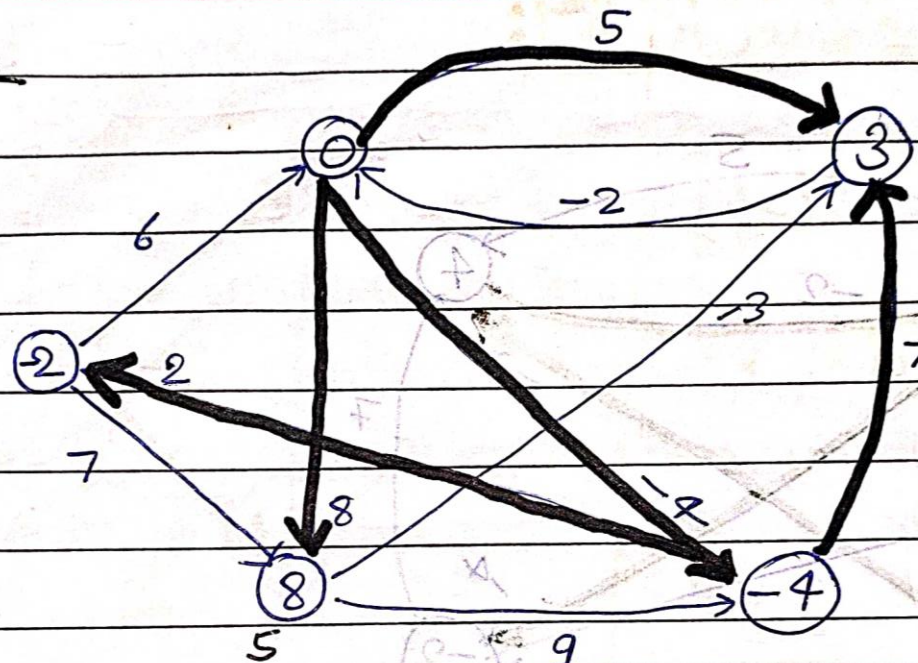
Ques:-



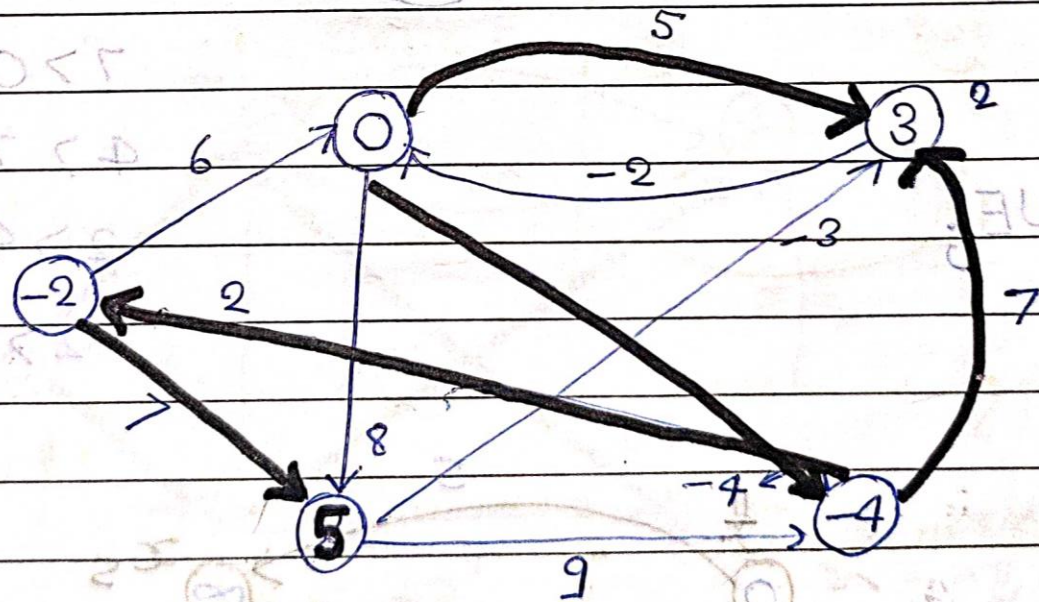
$i=1$



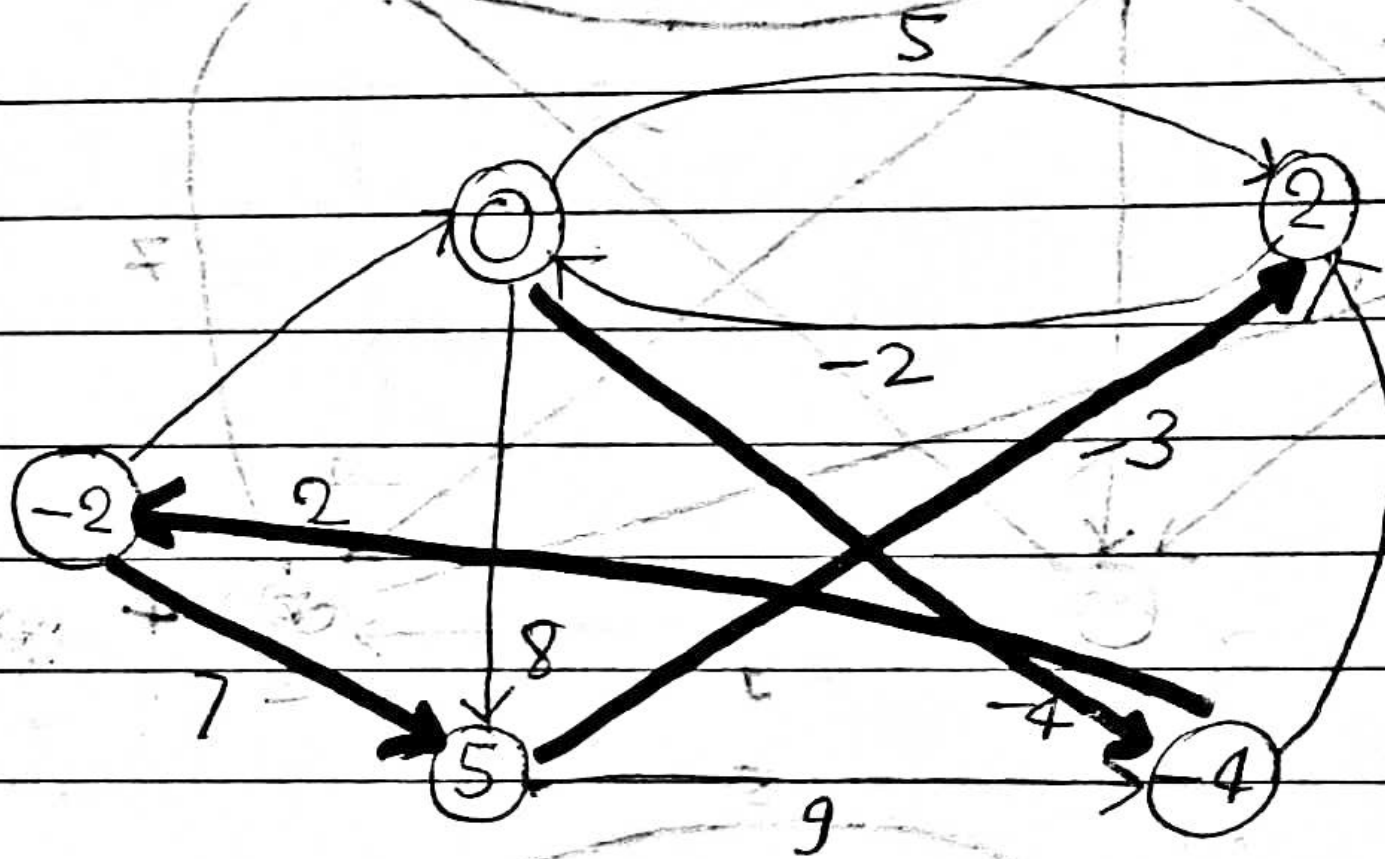
$i=2$



$i=3$



$i=4$



$5 > 7 - 2$ false

DIJKSTRA ALGORITHM:

Dijkstra algorithm solves the single source shortest path problem on a weighted directed graph from the case in which all edge weight are non-negative.

Running time $\Rightarrow O(E \log V)$

DIJKSTRA (G, w, s)

1. Initialize single source (G, s)

2. $S \leftarrow \emptyset$

3. $Q \leftarrow V[G]$

4. while $Q \neq \emptyset$

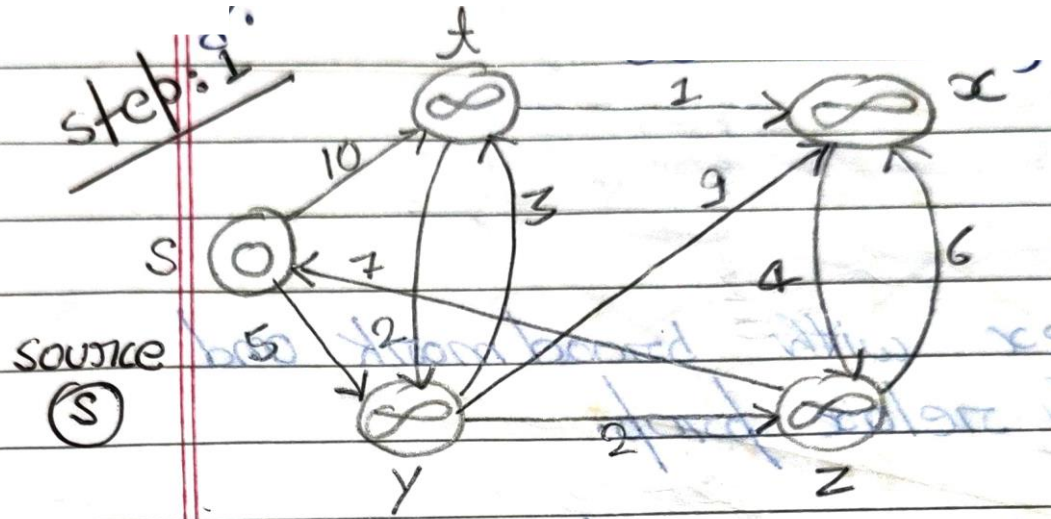
5. do $U \leftarrow \text{Extract_Min}(Q)$

6. $S \leftarrow S \cup \{U\}$

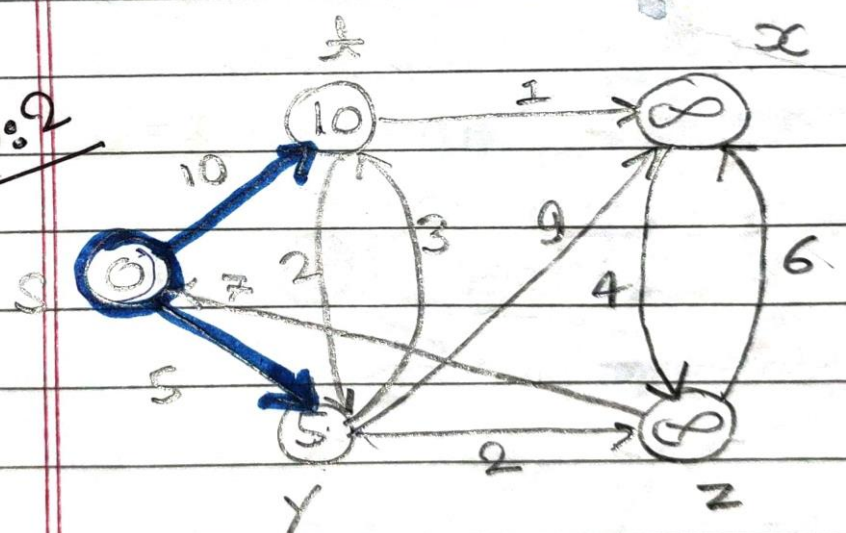
7. for each vertex $v \in \text{Adj}[U]$

8. do Relax (U, v, w)

step:1



step:2

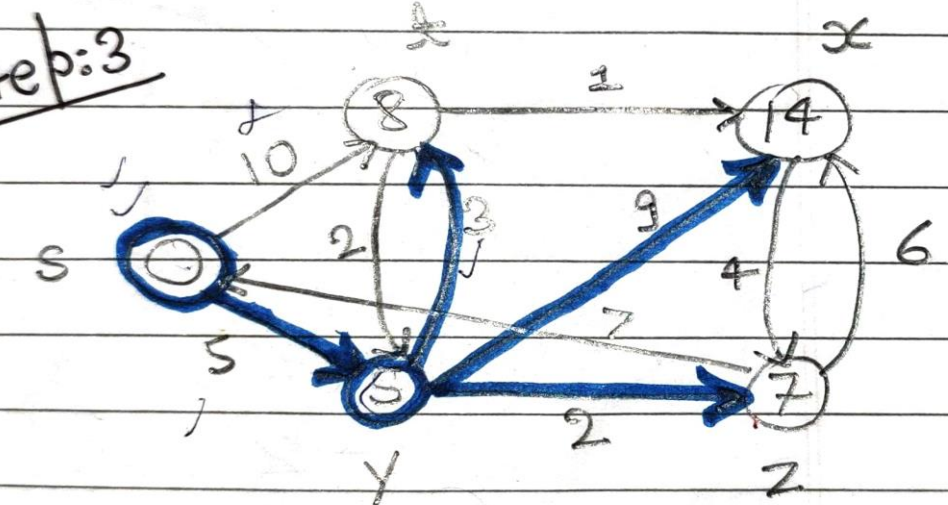


$S = \{\phi\}$

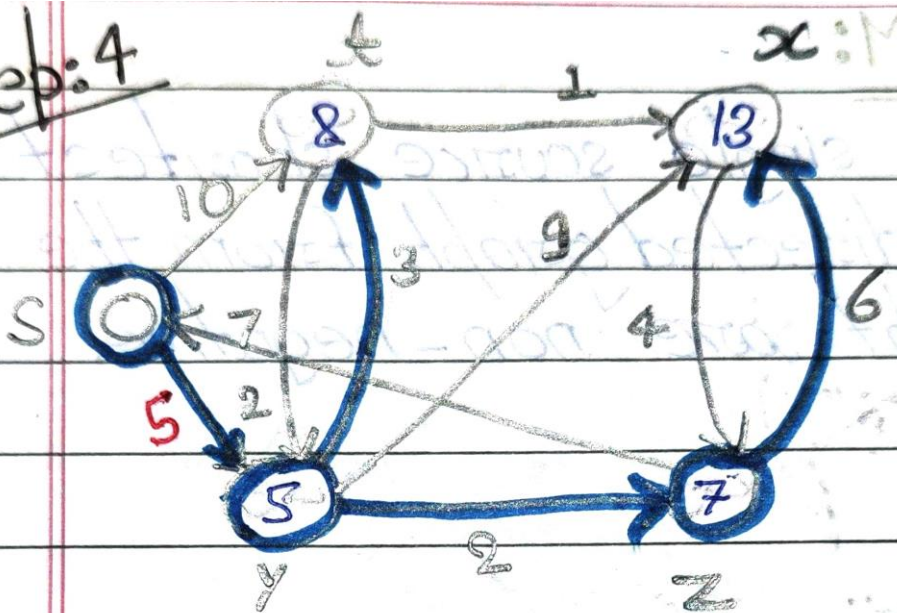
$Q \leftarrow \{s, t, x, y, z\}$

	s	t	x	y	z	
-	∞	∞	∞	∞	∞	$\Rightarrow S = \{s\}$
-	10	∞	5	∞	∞	$\Rightarrow S = \{s, y\}$
-	8	14	-	7	∞	$\Rightarrow S = \{s, y, z\}$
-	8	13	-	-	∞	$\Rightarrow S = \{s, y, z, t\}$
-	-	9	-	-	∞	$\Rightarrow S = \{s, y, z, t, x\}$
-	-	-	-	-	-	$\Rightarrow S = \{s, y, z, t, x\}$

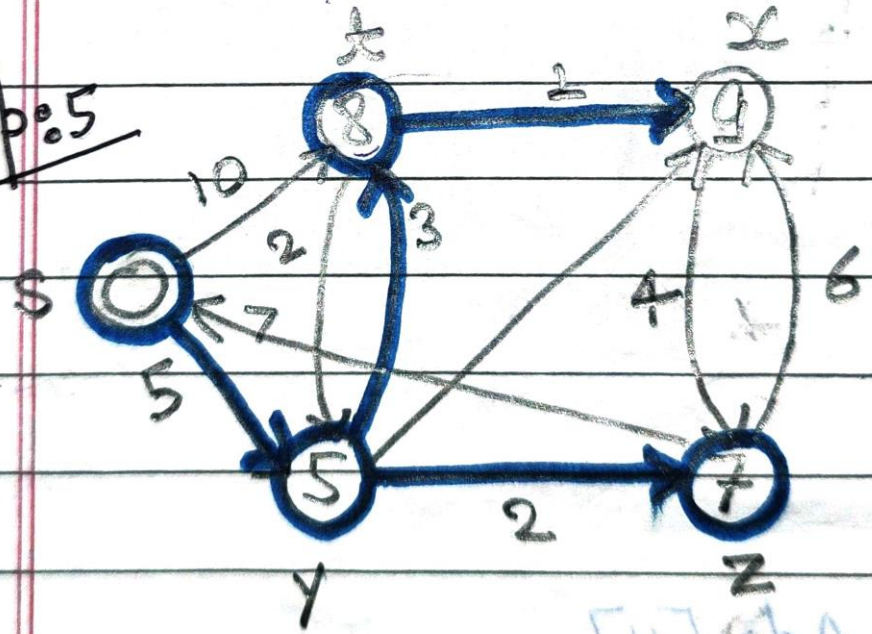
step:3



step: 4



step: 5



step: 6

